BÁO CÁO THỰC HÀNH HỆ ĐIỀU HÀNH - GVHD: Thân Thế Tùng

Họ và tên: Nguyễn Hoàng Thanh Tâm
Mã số Sinh viên: 25521638
Lớp: IT007.R117

HỆ ĐIỀU HÀNH
BÁO CÁO LAB 03

TÓM TẮT BÁO CÁO

(Có làm thì tick vào ô tương ứng)

1. CLASSWORK

	EX 1	EX 2	EX 3	EX 4	EX 5	EX 6
Phương pháp	☒	☒	☒	☒	☐	☐
Minh chứng	☒	☒	☒	☒	☐	☐
Giải thích	☒	☒	☒	☒	☐	☐

2. HOMEWORK

	A	B	C
Method	☒	☐	☐
Proof	☒	☐	☐
Explanation	☒	☐	☐

Tự đánh giá: Có tìm hiểu,nghiên cứu các hàm mới. Bổ sung link github về các bài code đã làm trên lớp và ở nhà.

3. MÃ NGUỒN

- Github Repository: https://github.com/thanhtamMalus/HDH/tree/main/Lab3

REPORT

1. CLASSWORK

Bài 1.

Ví dụ 3-1:

Truyền vào tham số 1 2 3 sau câu lệnh ./test_fork, argc sẽ nhận được là 4 tham số.

Fork() tạo ra tiến trình con, pid là kết quả trả về cho chương trình biết tiến trình này là cha hay con

pid>0 : tiến trình cha → in ra Parents, PID và PPID của tiến trình.

Argc - 1 = 3 → sẽ in thông báo rằng có 3 arguments

pid==0 : tiến trình con → in ra children, PID và PPID của tiến trình con

Sau đó xét duyệt i từ 1 → 3 để in ra các giá trị

argv[1]=1

argv[2]=2

argv[3]=3

home > malus > HDH > Lab3 > vd3-1 > C test_fork.c

```c
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
int main(int argc, char *argv[])
{
__pid_t pid;
pid = fork();
if (pid > 0)
{
printf("PARENTS | PID = %ld | PPID = %ld\n",
(long)getpid(), (long)getppid());
if (argc > 2)
printf("PARENTS | There are %d arguments\n",
argc - 1);
wait(NULL);
}
if (pid == 0)
{
printf("CHILDREN | PID = %ld | PPID = %ld\n",
(long)getpid(), (long)getppid());
printf("CHILDREN | List of arguments: \n");
for (int i = 1; i < argc; i++)
{
printf("%s\n", argv[i]);
}
}
exit(0);
}
```

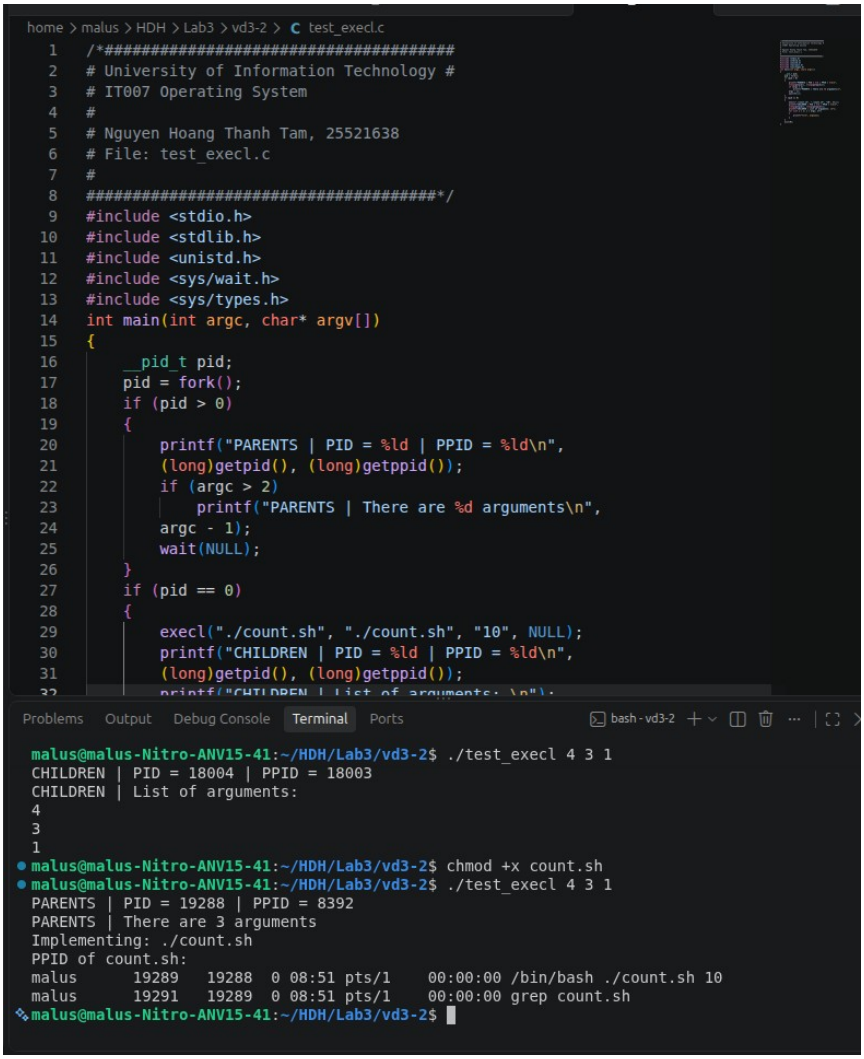
Problems Output Debug Console Terminal Ports bash - vd3-1

```
malus@malus-Nitro-ANV15-41:~/LTHT$ cd /HDH/Lab3/vd3-1
bash: cd: /HDH/Lab3/vd3-1: No such file or directory
malus@malus-Nitro-ANV15-41:~/LTHT$ cd /HDH/Lab3/
bash: cd: /HDH/Lab3/: No such file or directory
malus@malus-Nitro-ANV15-41:~/LTHT$
malus@malus-Nitro-ANV15-41:~/LTHT$ cd ~/HDH/Lab3/vd3-1
malus@malus-Nitro-ANV15-41:~/HDH/Lab3/vd3-1$ gcc test_fork.c -o test_fork
malus@malus-Nitro-ANV15-41:~/HDH/Lab3/vd3-1$ ./test_fork 1 2 3
PARENTS | PID = 13682 | PPID = 8392
PARENTS | There are 3 arguments
CHILDREN | PID = 13683 | PPID = 13682
CHILDREN | List of arguments:
1
2
3
malus@malus-Nitro-ANV15-41:~/HDH/Lab3/vd3-1$
```

Ví dụ 3-2:

Lệnh execl() là lệnh thay thế các dòng lệnh sau thành lệnh mới trong () khiến cho tiến trình con bị thay đổi so với ví dụ 3-1.

File count.sh chính là file được thay thế vào đoạn thực thi tiếp theo đó.



Ví dụ 3-3:

Khác với execl(), system() không xóa dữ liệu hay thay đổi các dòng code sau nó mà tạo ra một tiến trình trung gian, chạy xong sẽ tiếp tục các đoạn code bên dưới:



home > malus > HDH > Lab3 > vd3-3 > C test_system.c

```c
/*###################################
# University of Information Technology #
# IT007 Operating System
#
# Nguyen Hoang Thanh Tam, 25521638
# File: test_system.c
#
###################################*/
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/types.h>
int main(int argc, char* argv[])
{
    printf("PARENTS | PID = %ld | PPID = %ld\n",
    (long)getpid(), (long)getppid());
    if (argc > 2)
        printf("PARENTS | There are %d arguments\n", argc - 1);
    system("./count.sh 10");
    printf("PARENTS | List of arguments: \n");
    for (int i = 1; i < argc; i++)
    {
        printf("%s\n", argv[i]);
    }
    exit(0);
}
```

Problems Output Debug Console Terminal Ports bash - vd3-3

```
malus@malus-Nitro-ANV15-41:~/HDH/Lab3/vd3-3$ ./test_system 1 4 2
malus@malus-Nitro-ANV15-41:~/HDH/Lab3/vd3-3$ gcc test_system.c -o test_system
malus@malus-Nitro-ANV15-41:~/HDH/Lab3/vd3-3$ ./test_system 1 4 2
PARENTS | PID = 21097 | PPID = 8392
PARENTS | There are 3 arguments
Implementing: ./count.sh
PPID of count.sh:
malus      21098   21097  0 09:02 pts/1    00:00:00 sh -c -- ./count.sh 10
malus      21099   21098  0 09:02 pts/1    00:00:00 /bin/bash ./count.sh 10
malus      21101   21099  0 09:02 pts/1    00:00:00 grep count.sh
PARENTS | List of arguments:
1
4
2
malus@malus-Nitro-ANV15-41:~/HDH/Lab3/vd3-3$
```

Ví dụ: 3-4:

Tiến trình A:

Tạo và cấu hình vùng nhớ:

Sử dụng shm_open để tạo mới vùng nhớ chia sẻ tên "OS" và dùng ftruncate để định kích thước bộ nhớ.

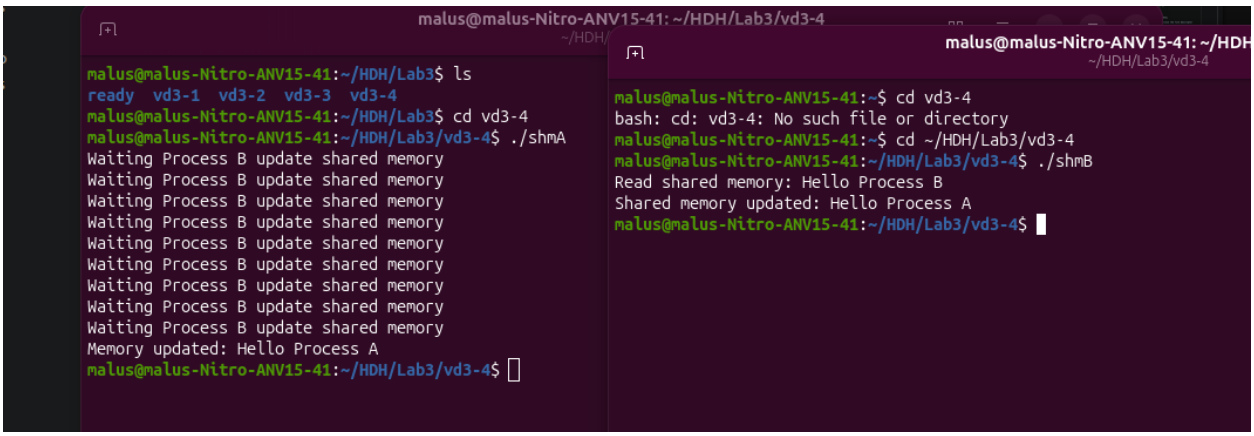
Ánh xạ và ghi dữ liệu: Dùng mmap để ánh xạ vùng nhớ rồi ghi chuỗi "Hello Process B" vào đó.

Chờ phản hồi: Dùng vòng lặp kiểm tra liên tục; chừng nào Tiến trình B chưa thay đổi nội dung, Tiến trình A sẽ tiếp tục chờ.

Hoàn tất: Khi Tiến trình B cập nhật dữ liệu mới ("Hello Process A"), Tiến trình A in kết quả ra màn hình và thực hiện hủy ánh xạ, đóng tài nguyên.

Tiến trình B thực hiện việc giao tiếp với tiến trình A thông qua mô hình Bộ nhớ chia sẻ.

Mở vùng nhớ: Sử dụng hàm shm_open để mở đối tượng bộ nhớ chia sẻ có tên "OS" đã được tạo bởi Tiến trình A.
Ánh xạ bộ nhớ: Sử dụng hàm mmap để ánh xạ vùng nhớ chia sẻ vào không gian địa chỉ của tiến trình với quyền đọc và ghi (PROT_READ | PROT_WRITE).
Đọc dữ liệu: Đọc và in ra màn hình nội dung hiện tại đang lưu trong vùng nhớ chia sẻ (do tiến trình A đã ghi vào trước đó).
Cập nhật dữ liệu: Ghi đè chuỗi "Hello Process A" vào vùng nhớ chia sẻ để gửi phản hồi ngược lại cho Tiến trình A.
Dọn dẹp tài nguyên: Sau khi tạm dừng 5 giây, tiến trình thực hiện hủy ánh xạ (munmap), đóng mô tả file (close), và gọi shm_unlink để xóa hẳn vùng nhớ chia sẻ khỏi hệ thống.



Bài 2:

- Phương pháp:

Dùng hàm fork() để tạo tiến trình con. Tiến trình cha wait đến khi con chạy xong.

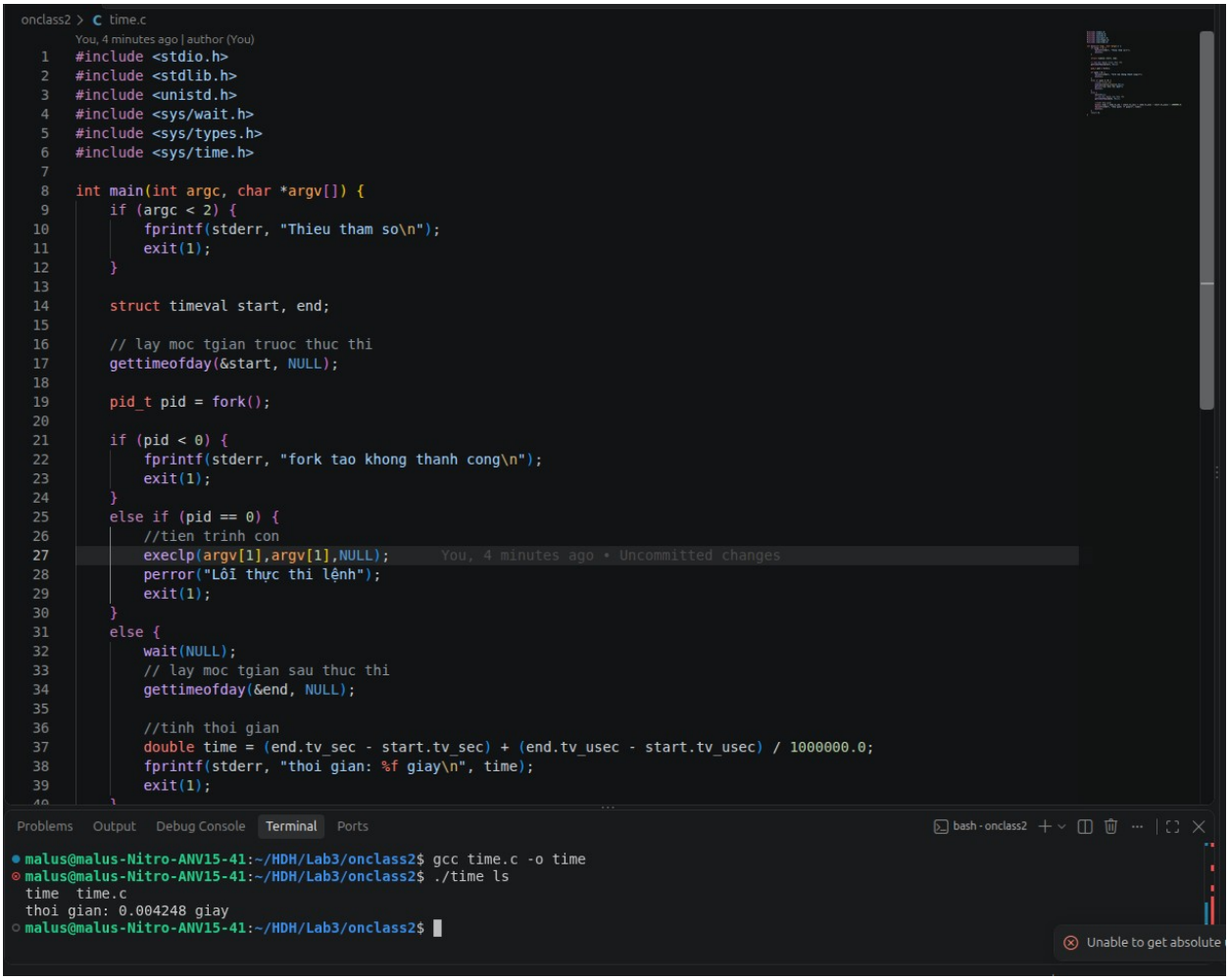
Dùng lệnh gettiemofday() để lấy thời gian start shell.

Tiến trình con thực hiện lệnh shell xong sang tiến trình cha.

Dùng lệnh gettiemofday() để lấy thời gian end shell

Tính thời gian thực hiện bằng cách là end trừ start.

- Minh chứng:



- Giải thích:

Bài 3:

- Phương pháp:

Hiển thị thông điệp khởi động: Sử dụng fprintf kết hợp với ký tự xuống dòng (\n) hoặc lệnh fflush(stdout) để ép bộ nhớ đệm (buffer) đẩy dữ liệu ra màn hình ngay lập tức, tránh tình trạng bị trôi thông báo do cơ chế đệm của C.

Quản lý tín hiệu (signal): Đăng ký hàm bắt tín hiệu SIGINT (khi người dùng nhấn tổ hợp phím Ctrl+C) thông qua hàm signal().

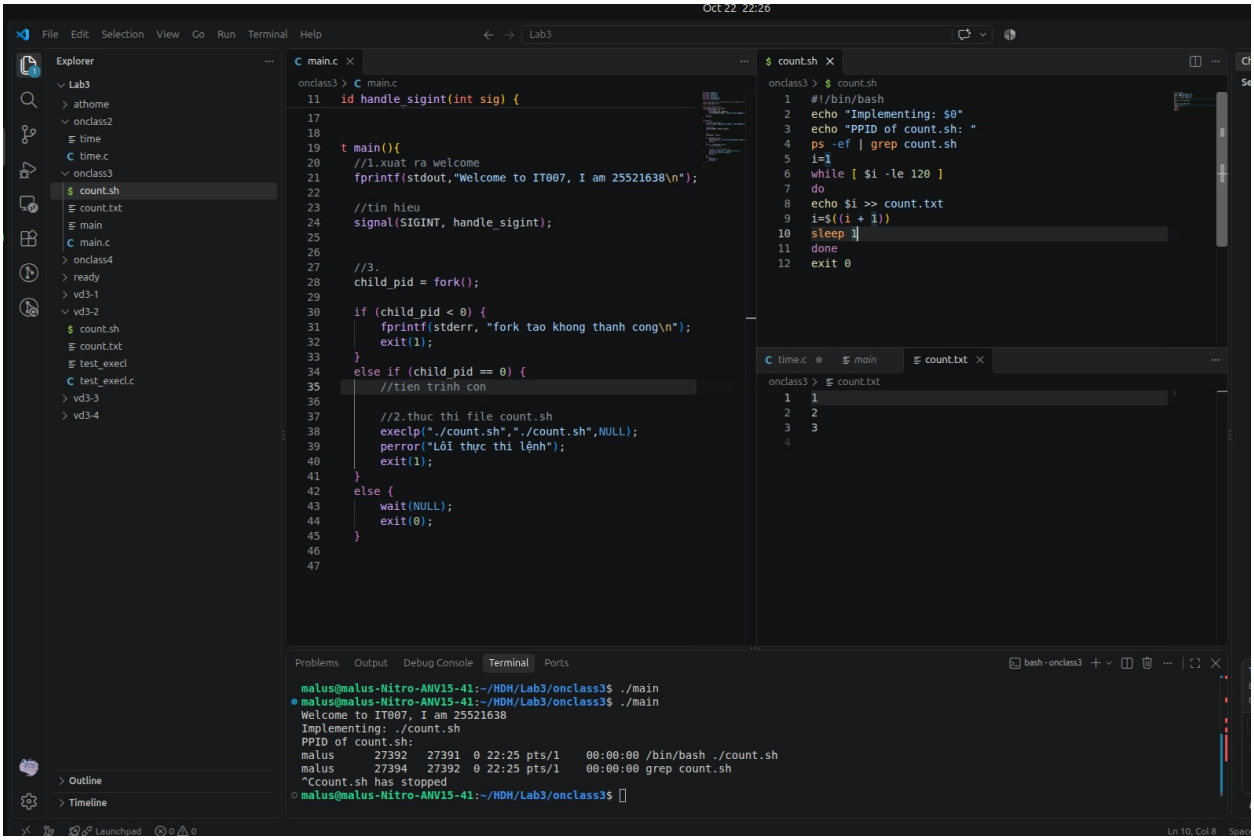
Khi bắt được tín hiệu, tiến trình cha sẽ thực hiện dọn dẹp hoặc tiêu diệt tiến trình con đang chạy ngầm.

Tạo và thực thi tiến trình (fork & execlp):Dùng hàm fork() để nhân bản tiến trình thành tiến trình cha và tiến trình con.

Trong tiến trình con, gọi hàm execlp() để nạp và thực thi file script ./count.sh với tham số đếm là 120.

Trong tiến trình cha, dùng hàm wait() để chờ tiến trình con hoàn tất công việc.

- Minh chứng:



- Giải thích:

execlp: Tự động tìm kiếm file thực thi (./count.sh) dựa vào biến môi trường PATH, thay thế không gian địa chỉ của tiến trình hiện tại bằng chương trình mới và truyền các tham số dòng lệnh vào ("120").

Cơ chế Ctrl+C (SIGINT): Khi tiến trình đang chạy foreground và bạn nhấn Ctrl+C, hệ điều hành sẽ phát tín hiệu ngắt. Hàm xử lý tín hiệu (handle_sigint) sẽ được kích hoạt để gửi tín hiệu SIGKILL tiêu diệt tiến trình con (count.sh) đang chạy ngầm và in ra dòng thông báo count.sh has stopped theo đúng yêu cầu.

Bài 4:

- Phương pháp giải quyết

Để giải quyết bài toán Producer - Consumer sử dụng Shared Memory và đồng bộ hóa tiến trình, chúng ta áp dụng các kỹ thuật sau:

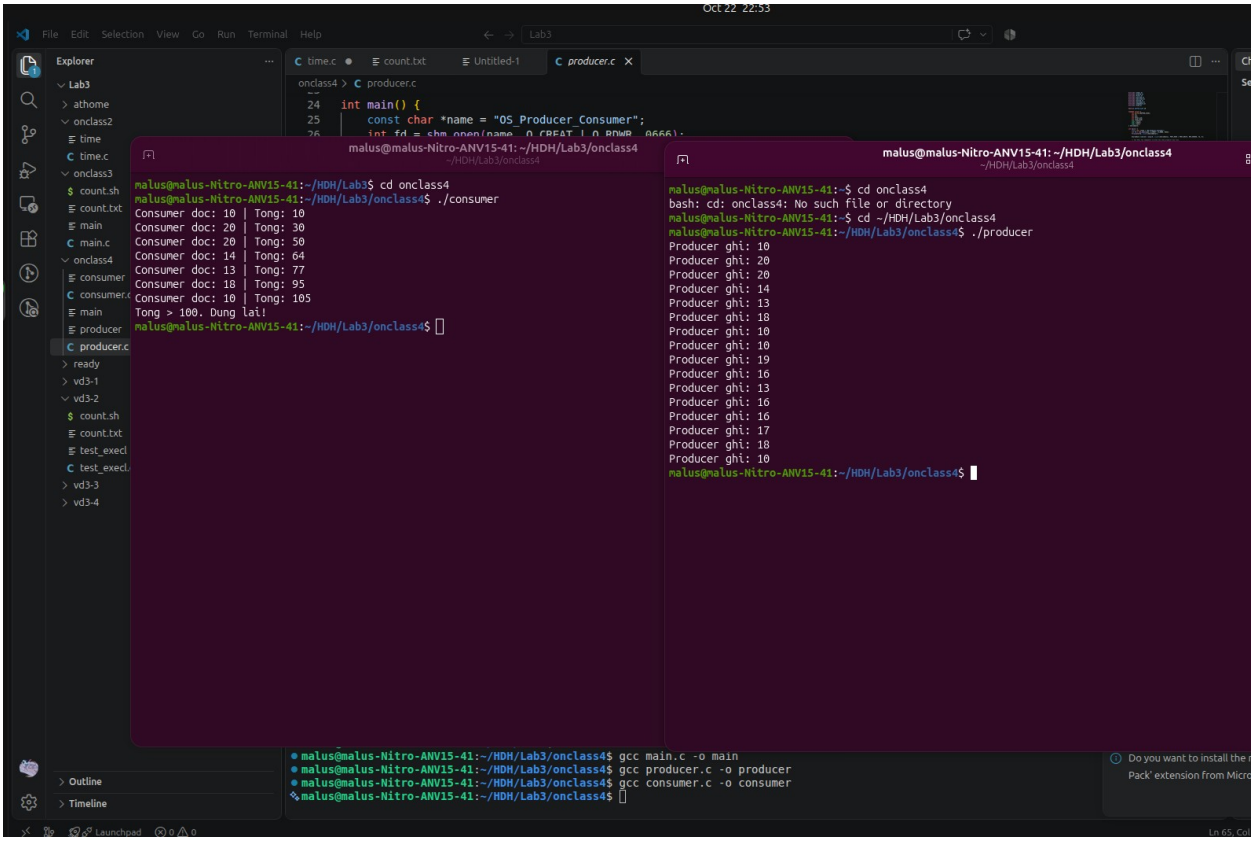
Shared Memory (shmget, shmat): Tạo một vùng nhớ chia sẻ giữa tiến trình cha và tiến trình con để lưu trữ cấu trúc buffer (mảng kích thước 10, con trỏ đọc/ghi, biến tổng và cờ dừng).

Semaphore tiến trình (sem_init với PTHREAD_PROCESS_SHARED): Dùng 3 semaphore để đồng bộ hóa:

+ mutex: Đảm bảo quyền truy cập độc quyền vào buffer (khóa tương hỗ).
+ empty: Đếm số ô trống trong buffer (khởi tạo bằng 10).
+ full: Đếm số ô đã có dữ liệu trong buffer (khởi tạo bằng 0).

Điều kiện dừng: Tiến trình Consumer cộng dồn giá trị đọc được vào total_sum. Khi total_sum > 100, Consumer bật cờ stop và giải phóng semaphore để Producer không bị kẹt, sau đó cả hai tiến trình cùng kết thúc.

- Minh chứng:



- Giải thích:

Cấu trúc SharedData: Gom tất cả các biến chung (mảng buffer, con trỏ in/out, biến total_sum, cờ stop_flag và các semaphore) vào một struct duy nhất đặt trong shared memory để cả cha và con đều truy cập trực tiếp được.

Semaphore pshared = 1: Tham số thứ hai trong hàm sem_init được truyền số 1 để đảm bảo semaphore này có hiệu lực đồng bộ giữa các tiến trình khác nhau (khác với thread nội bộ).

Cơ chế hoạt động Bounded-Buffer: Producer dùng sem_wait(&empty) để kiểm tra xem buffer còn chỗ trống không, sau đó dùng sem_wait(&mutex) để khóa vùng nhớ khi ghi. Ngược lại, Consumer dùng sem_wait(&full) để kiểm tra xem có dữ liệu để đọc hay không rồi mới lấy ra.

Thoát vòng lặp an toàn: Khi tổng vượt quá 100, Consumer đặt shared->stop_flag = 1 và thực hiện sem_post để giải phóng các semaphore đang bị khóa, giúp Producer thoát khỏi trạng thái chờ và kết thúc chương trình sạch sẽ không bị treo (deadlock).

2. HOMEWORK

- Phương pháp:

Nhận một số nguyên dương từ tham số dòng lệnh (argv), kiểm tra xem nó có hợp lệ không (phải > 0).

Thuật toán Collatz:

Nếu n chẵn: n = n / 2.

Nếu n lẻ: n = 3*n + 1.

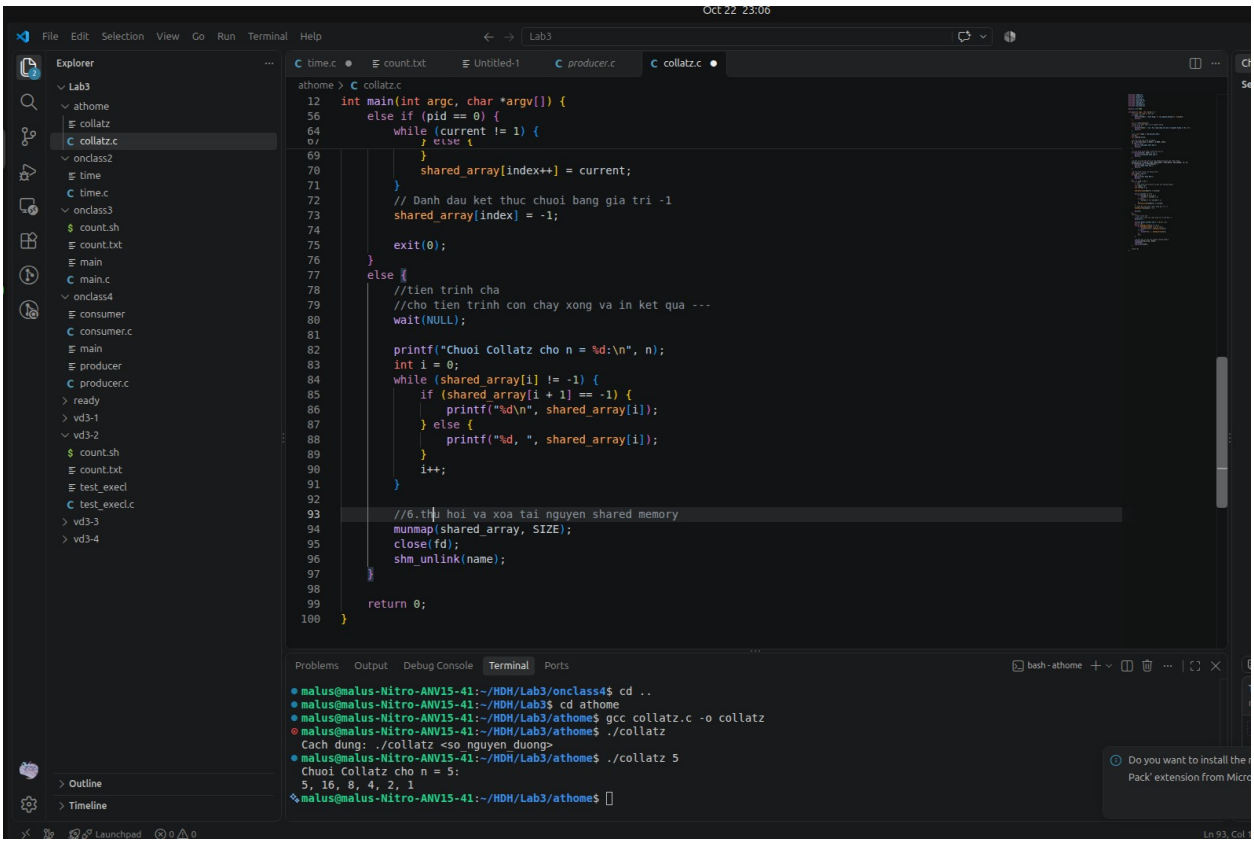
Lặp lại cho đến khi n = 1.

-> Sử dụng Shared Memory

Tiến trình con sẽ thực hiện tính toán chuỗi Collatz, ghi các số vào mảng nằm trong vùng nhớ chia sẻ.

Tiến trình cha sẽ wait đến khi tiến trình con chạy xong, sau đó đọc dữ liệu từ vùng nhớ chia sẻ để in toàn bộ chuỗi kết quả ra màn hình.

-Minh chứng:



-Giải thích:

Kiểm tra đầu vào: Chương trình kiểm tra argc < 2 để đảm bảo người dùng có nhập tham số và dùng atoi kết hợp điều kiện n <= 0 để chặn các giá trị không hợp lệ (số âm hoặc số 0) đúng theo yêu cầu ôn tập.

Thiết lập Shared Memory: Dựa hoàn toàn vào các hàm chuẩn POSIX: shm_open: Tạo đối tượng vùng nhớ chia sẻ chung với tên là "OS_Collatz_Shm". ftruncate: Cấp phát dung lượng kích thước 4096 bytes cho vùng nhớ.

mmap: Ánh xạ vùng nhớ đó vào không gian con trỏ số nguyên shared_array để cả tiến trình cha và con cùng truy cập được dưới dạng mảng.

Tiến trình con tính toán: Nhận giá trị n, thực hiện vòng lặp áp dụng công thức Collatz (nếu chẵn chia 2, nếu lẻ nhân 3 cộng 1) và lưu lần lượt các số tìm được vào shared_array. Ký tự -1 được đặt ở cuối mảng để làm tín hiệu đánh dấu kết thúc.

Tiến trình cha hiển thị và dọn dẹp: Tiến trình cha gọi wait(NULL) để đứng chờ tiến trình con hoàn tất công việc. Sau khi con xong, cha duyệt qua mảng trong vùng nhớ chia sẻ để

in toàn bộ chuỗi ra màn hình định dạng ngăn cách bởi dấu phẩy, cuối cùng tiến hành hủy ánh xạ và xóa đối tượng shared memory bằng shm_unlink để tránh tràn hoặc chiếm dụng.